MetricGuard Product and Engineering Plan

KPI reconciliation and definition drift

Prepared for Aditya Beecha | 22 September 2026 | Version 1.0

Build a SQL reconciliation workbench first. Compare existing KPI definitions on a frozen dataset, show the records and rule changes behind the gap, and track changes against a reviewed definition. Add broader BI integrations after the core calculation is demonstrably reliable.

This pack turns the starting concept into a bounded product and engineering specification. It covers the PRD, user experience, architecture, data and API contracts, reconciliation method, security, evaluation, demo data and delivery plan. It is intended to guide implementation and review before application coding begins.

The recommendation preserves the strongest part of the idea: investigating inconsistent business answers when the underlying data is structurally valid. It corrects assumptions about equivalent metric names, missing filter context and unsupported causal explanations.

Decision	Recommended first release
Domain and input	Synthetic retail data and a declared DuckDB SQL subset
Core result	Same-snapshot comparison with row evidence and an ordered value bridge
Drift	Immutable definition versions and fixed-data replay
Human review	Confirm intended metric relationships and scoped baselines
AI	Optional later for metadata suggestions; no dependency in the core
Delivery	Local application with a reproducible investigation and honest limitations

Status: proposed specification. Worked-example arithmetic has been checked independently. No application implementation, customer validation, product benchmark or deployment is claimed.

Reading guide

Start with chapters 1 and 4 to understand the product and its calculation method. Read chapters 3, 5 and 6 before implementing the engine. Use chapters 8 to 10 as the acceptance and delivery contract. The editable companion folder includes the chapter sources, diagrams, fixture CSV, SQL examples and proposed JSON contracts.

1 Product refinement and PRD

2 User journeys and screen specifications

3 Metric contract and SQL support

4 Reconciliation and semantic drift method

5 Architecture and deployment

6 Data model and API contracts

7 Security and operational requirements

8 Demo data and evaluation plan

9 Backlog and delivery roadmap

10 Decisions risks and coding handoff

11 Sources and glossary

Sources are cited as S1 to S8 beside relevant capability statements and listed with direct links in chapter 11. Architecture choices and numerical targets are design recommendations. The synthetic example is deliberately small enough to verify by hand.

1 Product refinement and PRD

Recommended product

Build MetricGuard as a KPI reconciliation workbench with definition history. Its main job is to help an analyst answer four questions: are these numbers intended to mean the same thing, how do their definitions differ, how much does that difference matter on a controlled dataset, and what evidence supports the explanation?

The first deliverable should be a working investigation from imported SQL to a reproducible reconciliation report. This suits a data analyst portfolio because the substance is SQL interpretation, metric design, variance analysis, validation and communicating evidence. A polished interface supports that work; it does not replace it.

Recommended positioning: MetricGuard compares existing SQL KPI definitions, quantifies their differences on the same data snapshot, and tracks changes against an approved definition.

Longer term positioning: Discover related metrics across analytics tools and help teams reconcile their implementations before or alongside adopting a semantic layer. This is a product hypothesis, not a verified unserved market.

Corrections to the original concept

Original assumption	Correction	Product consequence
Similar names imply the same KPI	Net sales, booked sales and recognized revenue may be intentionally different	Suggest related concepts and ask an analyst to confirm the intended relationship
Completed equals not Cancelled	Not Cancelled can include Pending and other statuses; SQL NULL also needs explicit treatment	Show the exact predicate and affected statuses
Date attribution can be inferred from the example	The original snippets contain no date predicate	Require date field, period and timezone metadata or mark them unknown
Different totals imply a definition problem	Different snapshots, permissions, scope or source transformations can also change totals	Check comparability before attributing an impact
Matching totals prove matching definitions	Different formulas can agree on a dataset with no refunds or pending orders	Show definition status separately from observed value status
An LLM can explain the gap	It can invent causes and percentages	Compute evidence first; use templates for the initial release
Contributions are inherently unique	Refunds, filters and date rules can interact	Declare the decomposition method and its order
All BI formulas are easy to translate	DAX depends on model context; spreadsheets have their own evaluation behavior	Start with SQL and add explicitly bounded adapters
There should be one canonical Revenue	Several definitions can be valid for different business purposes	Approve definitions within a stated scope and allow intentional variants

Original assumption	Correction	Product consequence
A changed definition tells when meaning changed	Import timestamps only show when MetricGuard observed a change	Separate observed time, source change time and business effective time

Use single quotes for SQL string literals. Label the finance example as an illustrative paid net sales definition; payment alone does not establish an accounting revenue recognition policy.

Problem and users

The immediate problem is repeated manual investigation when reports with apparently related KPI names disagree. The work includes finding the queries, resolving hidden filters, checking refresh cutoffs and identifying the rows that contribute to the gap. Anomaly detection can signal a change without establishing why it happened.

The primary user is a data or BI analyst investigating a specific discrepancy. A second role, the metric owner, decides what the KPI should mean and whether a difference is intentional. A reviewer consumes the evidence and checks the proposed resolution. In the local portfolio release one person may act in all three roles; the interface must not imply independent approval occurred.

The best initial setting is one online retail business with an order-level extract and several SQL reports. This keeps the business meaning understandable while exposing realistic refunds, order statuses and date attribution issues. Avoid presenting that setting as representative of all industries.

Job to be done: When two revenue reports disagree, help me isolate the definition differences on comparable data, show the contributing records, and document the accepted interpretation so that the same dispute is easier to resolve next time.

Existing alternatives and differentiation

Approach	Established purpose	MetricGuard design implication
dbt Semantic Layer	Central definition and querying of metrics using MetricFlow [S1]	Complement adoption by investigating existing definitions and exporting an approved specification
Soda observability and contracts	Monitor data behavior and verify declared expectations [S2]	Reuse the expectation concept but make the investigation begin with existing logic
Datafold data diff and monitors	Compare datasets and monitor changes [S3]	A data diff is adjacent; add metric intent, rule differences and evidence connecting rules to values
SQLGlot	Parse, inspect, transform and compare SQL structures [S4]	Use it as a component, not as evidence that all business meanings are understood
Manual SQL and spreadsheet reconciliation	Flexible investigation by an analyst	Reduce repeated setup while preserving the analyst's ability to inspect every calculation

These sources establish overlap. They do not establish that competing products cannot perform a particular investigation. The defensible portfolio contribution is the end-to-end workflow and its evaluated limits, not a claim to have invented metric governance.

Outcomes and success measures

The primary outcome is a completed, reproducible reconciliation case with a reviewed relationship between definitions. A successful investigation can conclude that the metrics are intentionally different or that there is insufficient evidence; it need not conclude that a report is wrong.

Measure	Proposed release gate	How to measure
Supported fixture correctness	All declared supported golden cases match expected results	Fixed SQL, data and expected values in version control
Unsupported behavior	No unsupported test is marked equivalent or fully explained	Negative and adversarial corpus
Additive reconciliation	Endpoint gap equals the sum of row contributions to one paisa	Independent decimal arithmetic oracle
Repeatability	Identical versions, data and configuration produce identical numeric evidence	Re-run test with stored hashes
Candidate suggestion usefulness	At least 90 percent precision on the top ranked held-out pairs	Report sample size, recall and pair labels; target only
Investigation usability	At least four of five pilot users complete the core task without help	Small formative pilot; disclose that it is not a population estimate
Analyst effort	Record completion time and wrong conclusions versus a manual baseline	Same task, counterbalanced order where practical; no savings claim before testing
Performance	Define an initial 30 second target for the standard 100000-row case	Fixed hardware, warm and cold runs; revise after a measured spike

Do not combine these into a single opaque trust score. The interface should separately show import coverage, context completeness, definition differences, numeric impact and review status.

Scope by release

Capability	MVP	Later increment
SQL definitions	Upload or paste the supported DuckDB SQL subset	Tested PostgreSQL and warehouse dialect adapters
Data	One immutable flat snapshot per run, CSV or Parquet input	Controlled warehouse snapshots and cross-source mapping
Matching	Deterministic candidate suggestions plus human review	Optional metadata embeddings if evaluation shows value
Metric types	SUM, COUNT and single-column COUNT DISTINCT	Ratios, averages, funnels and non-additive metrics
Explanation	Row contributions for SUM and ordered rule bridge	Exact Shapley attribution for a bounded set of valid rule switches

Capability	MVP	Later increment
Drift	Definition versions, explicit comparison and manual replay	Scheduled monitoring and repository integration
Governance	Local owner decision and scoped approved baseline	Authenticated roles, review workflow and notifications
Power BI	Product roadmap only; screenshots are not executable evidence	TMDL metadata import, then native DAX execution with full context
dbt	Compiled SQL may be pasted if it fits the subset	Manifest import with version-aware lineage and SQL handling
Excel	CSV export treated as data, without formula evaluation	Bounded formula metadata extraction before any calculation support
Deployment	Local application and reproducible synthetic demonstration	Hosted private pilot; public demo can use precomputed results

Explicit exclusions for the first release: arbitrary joins and CTEs, arbitrary DAX or Excel evaluation, automatic production corrections, universal SQL equivalence proofs, financial accounting certification, enterprise multitenancy, complex warehouse connectivity and an LLM-driven SQL execution agent.

Functional requirements

ID	Requirement	Acceptance condition
FR01	Import definitions and record provenance	Every definition retains original text, dialect, source label, version and import time
FR02	Parse and classify support	Every import is executable, metadata only, needs context, or unsupported, with a reason
FR03	Propose related metrics	Suggestions show the name and structural evidence; none is auto-approved
FR04	Confirm comparison context	Currency, grain, snapshot, period and scope checks run before execution
FR05	Compare definitions	Changes to amount expression, predicate and time field remain visible even when totals match
FR06	Execute a comparison	Both definitions run on the same frozen snapshot with a stored context
FR07	Explain supported gaps	Report endpoints, signed contributions, method, samples and any unexplained amount
FR08	Track definition drift	Preserve old and new definitions and compare them on a fixed snapshot
FR09	Record a reviewed baseline	Approval stores scope, rationale, version, effective time and actor
FR10	Show registered report dependencies	Every affected asset has a recorded dependency path and coverage label

ID	Requirement	Acceptance condition
FR11	Export the evidence	A readable report and JSON include enough identifiers to reproduce the run
FR12	Handle failure honestly	Parse failures, absent data, timeout and unsupported features cannot become green success states

Validation before expanding scope

Interview three to five analysts about an actual recent metric disagreement. Ask where the logic lived, which contexts were missing, what evidence resolved the dispute, and whether access to transaction data was possible. Ask for sanitized examples only. Record whether the problem is mostly definition inconsistency, data freshness, accounting policy, or ownership confusion.

Proceed with the SQL workbench if at least two independently described cases fit the supported workflow and users can supply comparable extracts. If all cases require inaccessible DAX models or unrelated warehouses, adjust the adapter strategy before claiming product fit. The portfolio build can still proceed with synthetic evidence while clearly labeling the validation gap.

2 User journeys and screen specifications

Primary investigation

An analyst opens a project and imports a dataset. The application shows the schema, row count, grain and validation failures. After confirming the business metadata, the analyst imports existing SQL definitions. MetricGuard suggests related definitions; the analyst decides whether they are expected to agree, intentional variants or unrelated.

The analyst selects two versions, chooses a common reporting period and runs the comparison. The result opens with the observed difference, definition differences and evidence coverage. The analyst can inspect the contributing rows and the ordered rule bridge, then record an explanation or an approved scoped definition. Exporting the report preserves the evidence without changing any source system.

A canonical definition is optional at the start. Existing logic is enough to begin; a reviewed baseline is a possible outcome of the investigation. The first-run form therefore asks for context needed to interpret the logic, not a complete semantic model written from scratch.

Six screens

Screen	Main content	Primary action	Important states
Project overview	Imports, reviewed pairs, open cases, definition changes and coverage	Start investigation	Empty project; imports incomplete; no open cases
Import and validate	Dataset schema, grain, source metadata, parse support and missing context	Confirm import	Rejected file; duplicate key; unknown currency; unsupported query

Screen	Main content	Primary action	Important states
Metric inventory	Definition name, source, family, version, support and owner	Review suggested pair	Candidate; intentional variant; rejected match
Comparison workspace	Context, two definitions, totals, differences and explanation tabs	Run comparison	Needs context; queued; running; partial; failed; complete
Definition history	Version timeline, structural changes, same-snapshot replay and recorded decisions	Compare versions	First import; cosmetic change; material change; approved change
Baseline and evidence	Scoped definition, known consumers, rationale and exported evidence	Record decision or export	Draft; approved locally; superseded; replay unavailable

Comparison workspace detail

The top line identifies the metric family, the exact two definition versions and the context. For example: Sales reporting, August 2026, INR, order grain, synthetic snapshot demo01. Below it, show Dashboard A at INR 104000000.00, Finance at INR 98000000.00, and Target minus baseline at INR -6000000.00.

Present four tabs: Definition differences, Value bridge, Evidence rows, and Provenance. Definition differences compares amount formula, business predicate, time field, missing-value behavior and scope. The value bridge shows the ordered steps with signed amounts. Evidence rows provide a paginated contribution table and can be sorted by absolute impact. Provenance provides version, snapshot and engine information for reviewers who need it.

The explanation should read naturally: "On the imported August snapshot, the finance definition is INR 6000000 lower. Applying refund treatment, then payment eligibility, then payment date explains the observed gap. This bridge uses that declared order." Every sentence containing a numeric result must be generated from the stored result, not composed from memory or an LLM estimate.

Status model

Use separate status fields instead of one red or green badge. A metric can have a definition difference while the currently observed values match.

Dimension	Values	Meaning
Relationship	Candidate, expected agreement, intentional variant, unrelated	Human interpretation of business purpose
Context	Comparable, needs context, not comparable	Whether a fair replay can run
Definition	Same supported representation, different, unknown	Structural assessment within the declared support boundary
Value	Equal on snapshot, within tolerance, different, not run	Observed numeric assessment

Dimension	Values	Meaning
Explanation	Complete ordered bridge, partial, row evidence only, unavailable	What the evidence supports
Review	Open, reviewed, baseline recorded, superseded	Workflow state, not proof of correctness

Do not label a candidate as a conflict until the relationship is expected agreement and the context is comparable. Intentional variants can still be compared for understanding but do not trigger an inconsistency alert. Unknown and unsupported require text labels and cannot be shown as passing checks.

Worked acceptance journeys

US01 Investigate revenue. Given the eight-row fixture and definitions A and Finance, when the analyst confirms a comparable context, the system returns INR -6000000.00 and exposes the five baseline-included orders. Four rows have nonzero contributions. The bridge and row totals independently reconcile.

US02 Avoid a false match. Given Net Sales and Recognized Revenue as names, a name suggestion may appear. The user must choose their relationship. Without confirming purpose and scope, the system cannot declare a conflict or pick one as correct.

US03 Detect a hidden definition difference. Given two different predicates and a dataset containing only completed orders, values may match. The result still shows the predicate difference and the label Equal on snapshot.

US04 Explain why a replay is blocked. Given an unknown timezone or missing date field, the comparison page lists the missing context with an edit action. It does not produce a numeric impact.

US05 Review a change. Given an imported new version that changes order date to payment date, the history page shows the old and new logic, when the change was observed, and a replay on the same dataset. It does not infer when a production report first changed unless source history provides that time.

US06 Work without an AI service. The entire investigation, export and baseline flow must work offline using deterministic suggestions and explanation templates.

Design and accessibility

Use a restrained analytics interface with a light background, clear tabular numbers and direct actions. Use INR in exports and allow Indian digit grouping in the UI. Keep the unrounded amount available beside compact crore or lakh formatting. Avoid animated counters and an overall trust percentage.

Use a waterfall for the additive value bridge and a table beside it containing every exact value. Use line charts only for genuine replay periods, not invented historical values. Color is supplementary: include status text and icons. Keyboard focus, labeled form controls, sufficient contrast, useful error messages and a readable narrow-screen layout are part of completion.

The overview's coverage should say "18 of 24 imported definitions executable" rather than imply all company reports were checked. Report impact should say "3 registered consumers potentially affected" when the graph contains only manually registered dependencies.

Errors and incomplete investigations

An upload that fails structural validation remains visible with its error and cannot become a dataset snapshot. A canceled run has a terminal canceled state. A timed-out run retains completed sub-results only as partial evidence. A missing original snapshot makes a previous result viewable but not replayable. If a user changes context, any displayed previous result must remain labeled with its original context or be cleared.

An export should contain the same status and limitations as the screen. Never transform partial findings into a definitive summary for presentation. A locally recorded owner name is a self-declared record, not authenticated sign-off.

3 Metric contract and SQL support

Definitions and context

A metric definition is a versioned implementation. A metric family is a reviewed grouping of related business concepts. A comparison context supplies the exact period, snapshot and interpretation needed to execute a definition. A baseline is an approved definition for a particular purpose and scope. Keep these objects separate.

The normalized representation, called the metric IR, is a typed intermediate representation of the supported SQL. It retains the aggregate, amount expression, eligibility predicate and time predicate as separate components. It is a restricted execution description, not an attempt to capture every analytical language.

Field group	Required information	Handling of missing information
Identity	Definition ID, immutable version, source label, original text and declared dialect	Reject an import without text or dialect
Meaning	Display name, purpose, family suggestion and owner label	Meaning remains unreviewed until confirmed
Population	Table, row grain, business predicate and scope	Block impact if grain or scope is unknown
Calculation	Aggregate, typed expression, NULL behavior and unit	Reject expressions outside the supported grammar
Time	Date column, inclusive start, exclusive end, business timezone and as-of cutoff	Ask the user to confirm; never invent a date field
Data	Snapshot ID, schema, content hash and mapping version	Require a valid immutable snapshot
Review	Relationship classification and optional scoped baseline	Allow investigation before approving a baseline
Provenance	Parser, IR, executor and configuration versions	Store versions in every run

Exact MVP SQL boundary

The first release accepts the DuckDB dialect only. An expression being parseable by SQLGlot is necessary but not sufficient. It must also fit the MetricGuard allowlist and be fully lowerable to the typed IR. SQLGlot documents best-effort transpilation behavior, so adapters must raise on unsupported translation and undergo tests [S4]. The MVP avoids cross-dialect transpilation.

Construct	MVP policy
Statement shape	Exactly one SELECT with one scalar aggregate and optional alias
FROM	Exactly one registered flat table, with an optional alias
Aggregation	SUM of approved numeric expression, COUNT star, COUNT DISTINCT one approved column
Numeric expression	Numeric columns and decimal literals, addition, subtraction, multiplication, COALESCE with typed numeric fallback
Predicates	AND, OR, NOT, parentheses, comparisons, IN with literals, IS NULL and IS NOT NULL
Period	Direct date or timestamp column with typed start and end parameters; half-open interval
Literals	Typed numbers, strings, dates and timestamps; single quotes for strings
Disallowed	Joins, CTEs, subqueries, windows, UDFs, GROUP BY, HAVING, UNION, ORDER BY, LIMIT and nested aggregates
Disallowed expressions	Division, arbitrary CASE, approximate functions, volatility such as current time or random
Non-query operations	DDL, DML, COPY, ATTACH, configuration and extension statements rejected
Table functions	All user-supplied file, URL and function-based scans rejected

The UI can supply a period to a query that lacks a date predicate only after the analyst selects a known column and explicitly confirms that this is the intended reporting context. Preserve the supplied original query and the resolved execution form. If a query has a conflicting hard-coded period, do not silently override it.

A parser should identify the period clause using the confirmed time field and the permitted predicate shape. Ambiguous clauses, multiple competing time fields or an inseparable OR containing the time restriction require manual clarification or an unsupported result. Parenthesized business predicates are allowed; arbitrary semantic rewriting is not.

Proposed definition example

The example JSON in `examples/metric_definition.json` proposes the API-level record. SQL remains the user-facing source. The server creates and validates the IR; users do not send a claimed parsed tree that the runner simply trusts.

Core example: metric `sales_reporting`, implementation `dashboard_a_v1`, dialect `duckdb`, aggregate `SUM(order_amount)`, predicate `status = 'Completed'`, time field `order_date`, grain `one row per order_id`, currency `INR`, date semantics `business_date`, timezone `Asia/Kolkata`. The interpretation

of order_amount as excluding tax must be stated in the dataset dictionary. If a different source includes tax, comparison is blocked until the difference is modeled.

Hash original text separately from normalized structure. Preserve meaningful details including NULL behavior, numeric types, timezone, case-sensitive identifiers and date boundaries. Only cosmetic changes with proven-safe normalization, such as whitespace or an output alias, may share a normalized fingerprint. Parser upgrades do not silently rewrite previous fingerprints.

Snapshot requirements

Each accepted snapshot contains one immutable analytical table and a manifest. The manifest includes content hashes, ingestion time, extraction cutoff, row count, schema types, primary key, business timezone, currency and any upstream transformation description. The hash represents bytes or a precisely defined canonical serialization; a row-count match is not a content identity check.

CSV ingestion requires explicit decimal, date and NULL handling. A blank string and SQL NULL are not automatically interchangeable. Invalid dates or decimal overflow fail validation or enter a clearly reported rejected-record file; they cannot disappear silently. Uniqueness of order_id must be checked before row-level evidence is enabled.

Currency conversion, exchange rates, source timezone conversion and hidden security filters are outside the MVP. Imported data must already use a documented single currency and a compatible scope. A normalized transformation made during ingestion must be recorded and versioned. The application must not silently clean away the reason reports disagree.

Comparability policy

Require the same snapshot, agreed logical table mapping, compatible grain, unit, currency, calendar, reporting window, business timezone and user-visible population scope. Amount rule, business predicate and date column are permitted differences to investigate; they must not be normalized away. A different declared security scope is a blocker rather than an unexplained contribution.

Three results are possible: comparable, needs context, and not comparable. Needs context means missing information can reasonably be supplied. Not comparable means the current data or scope cannot support the intended calculation. A future cross-source adapter will need explicit grain mapping, source completeness and reconciliation checks; the same column names are not enough.

Type and empty result semantics

Use DECIMAL for money and checked integer arithmetic for counts. Keep decimal strings in JSON so browser floating-point conversion does not alter stored evidence. INR outputs are displayed to two decimals, but calculations retain the configured precision until the final presentation stage. Reject overflow and narrowing casts.

SQL SUM ignores NULL terms and returns NULL when there are no non-NULL terms. COUNT star counts rows; COUNT DISTINCT ignores NULL values. Preserve these behaviors unless the original definition explicitly uses COALESCE. A row where a SUM term is NULL contributes no numeric amount, but it must still be flagged in the evidence. If either scalar endpoint is NULL, do not calculate an arithmetic gap by treating it as zero. Show an empty or undefined result.

For a money comparison, propose a one-paisa numeric tolerance. Business materiality is a separate threshold set by a user. For relative differences, use baseline magnitude as the denominator and display the direction separately. If the baseline is zero, the percentage is undefined. For rates in a future release, distinguish percentage points from percent change.

Integration support levels

Every adapter exposes four independent capabilities: import metadata, parse logic, execute natively, and reconcile quantitatively. Supporting the first does not imply the others.

Power BI TMDL provides model metadata [S5]. A later importer should preserve measures, referenced columns, relationships and available provenance. DAX measures also rely on filter context propagated through model relationships [S6]. Numeric reconciliation therefore requires the actual model, equivalent identity and security scope, the exact filter context and refresh state. Native execution is preferable to claiming a general DAX-to-SQL converter.

For dbt, use version-aware manifest parsing and compiled SQL when present. Compiled content is not guaranteed for every node [S7]. Never execute imported Jinja or macros on the MetricGuard server. Lineage can identify dependencies, but a model is not automatically a KPI definition. Identify the specific aggregate query or metric definition to compare.

For Excel, formula text and cached values are separate evidence. A later importer must distinguish named ranges, workbook links, calculation settings and stale cached results. Until a bounded evaluator is tested, treat formulas as metadata and CSV exports as data only.

4 Reconciliation and semantic drift method

What the engine can establish

The engine can establish structural differences within its supported grammar and observed numeric differences on a specified snapshot. It cannot prove that arbitrary SQL programs are equivalent, infer a company's accounting policy from a name, or establish real-world causation from a data comparison.

Keep three kinds of evidence distinct: definition evidence, execution evidence and review evidence. Definition evidence says which rules differ. Execution evidence says what happens when the rules are run on frozen data. Review evidence records whether a knowledgeable person considers the differences intentional.

Pipeline

1. Preserve the original definition and source metadata.
2. Parse with the declared dialect and bind column names to the known schema.
3. Validate the entire syntax tree against the supported grammar.
4. Lower the definition into the typed metric IR and record unsupported features.
5. Suggest related definitions using metadata and structural features.
6. Ask the analyst to confirm expected agreement or intentional variation.
7. Check that the comparison context and snapshot are suitable.
8. Run the original supported endpoints on that snapshot using the validated execution form.

9. Calculate the structured rule diff and, where supported, the value bridge and row evidence.

10. Persist immutable results, limitations and provenance before rendering an explanation.

The engine must revalidate the IR in the runner. An API validation pass does not justify trusting an arbitrary stored job payload. Results become complete only after endpoint and reconciliation invariants pass.

Candidate matching

Start with a transparent heuristic. Normalize names conservatively, use a small reviewed alias dictionary, and rank pairs using shared source columns, table, aggregate family, date concept, unit and purpose description. Name similarity alone is insufficient. Names containing gross, net, booked or recognized should preserve those qualifiers.

The ranking score is a prioritization heuristic, not a probability. Display reasons such as "same amount column and currency; different status predicate." Persist confirmed positive and negative pair labels so the same suggestion is not repeatedly pushed. Do not merge metric families transitively just because A resembles B and B resembles C.

If embeddings are added later, send only approved metadata, include negative examples, and measure precision and recall against a frozen held-out pair set. The model may suggest a relationship; it cannot approve it, supply execution SQL or invent impact numbers. A metadata-only model outage must not stop a supported comparison.

Structural comparison

Compare aggregate, expression, eligibility predicate, time field, time boundaries, NULL treatment, units and scope. Report a known difference even if its financial effect is zero on the selected data. Whitespace, comments and an output alias should not create a business-meaning alert when their normalization is safely established.

Boolean expressions require SQL's three-valued NULL semantics. Do not simplify `status <> 'Cancelled'` into an assumed list of allowed statuses. Do not sort or rewrite every expression indiscriminately; numeric types, floating-point operations, collation and NULL behavior can affect results.

The output Same supported representation means identical validated IR under the stored normalization policy. Equal on snapshot means two execution results agree in that context. Neither label is a universal mathematical proof for all possible databases.

Endpoint calculation and row contributions

For a supported additive SUM, define each row's contribution as its amount expression if the row passes that definition's predicate and period, and zero if it does not. Preserve NULL-term flags separately and require valid non-NULL aggregate endpoints before computing a bridge.

For each order, the signed contribution is target contribution minus baseline contribution. On a unique order_id grain, the sum of these contributions must equal target total minus baseline total. Display baseline-only, target-only, included-in-both and included-in-neither membership, and the amount difference for rows included in both.

This membership partition is mutually exclusive, which prevents double counting. It is a record-level reconciliation, not a unique assignment to business causes. The same order can be affected by more than one changed rule. A contribution table and a rule bridge therefore answer different questions.

For COUNT star, report membership changes and exact count differences. For COUNT DISTINCT customer_id, compare distinct membership sets and use the distinct entity as the evidence unit. Do not distribute a customer's count contribution over its orders or sum distinct counts across segments. The MVP may show ordered rule effects for count metrics, but full money row-contribution charts apply only to SUM.

Ordered rule bridge for the MVP

Use a fixed documented sequence for the retail case: amount expression, then eligibility predicate, then time attribution. The start uses all baseline rules. Each intermediate definition replaces exactly one supported rule group with its target counterpart; the endpoint must reconstruct the target IR exactly.

Each contribution is the new intermediate total minus the preceding total. The telescoping sum then equals the endpoint difference if every intermediate is valid and evaluated. Store the intermediate definition, value, query hash and rule name. The UI must show the order because interactions can move amounts between steps.

Do not break a logically indivisible rule into arbitrary fragments. If a new amount expression requires a join absent from the old definition, or a hybrid rule creates an invalid scope, stop that bridge. Report the valid endpoints and supported row evidence, with a reason that rule attribution is unavailable. Never describe a partial bridge as complete.

When every supported rule has been applied but the reconstructed target differs, classify this as an internal reconciliation failure. Suppress a complete explanation and retain the endpoint gap as unexplained. Residual equals observed gap minus the sum of reported bridge contributions. A residual is a diagnostic, not a cause named Other.

The corrected revenue example

The synthetic fixture in chapter 8 was constructed to produce the original headline values. It does not represent an actual company. The order_amount field excludes tax and shipping, and refunds are already expressed at order grain as of a fixed cutoff. This simplified refund model is not a financial ledger.

Bridge step	Total INR	Change INR	Interpretation
Dashboard A	104000000.00	Start	Completed orders by order date
Apply refunds	99680000.00	-4320000.00	Use order amount minus refund amount
Apply Paid eligibility	99260000.00	-420000.00	Replace Completed with Paid eligibility
Apply payment date	98000000.00	-1260000.00	Attribute the period using payment date
Finance endpoint	98000000.00	-6000000.00 total	Illustrative paid net sales

In Indian units the totals are 10.4 crore and 9.8 crore; the gap is 60 lakh. For this declared order, the contributions are 43.2 lakh, 4.2 lakh and 12.6 lakh, corresponding to 72 percent, 7 percent and 21 percent of the signed gap. Another valid order can allocate interactions differently. These percentages must be calculated from the actual run, not stored as an explanation template.

Dashboard B using not Cancelled produces 10.47 crore in this fixture because it additionally includes a pending order. The original assumption that A and B both produce 10.4 crore is not generally valid.

Percentages should be secondary to currency amounts. Opposing contributions can produce negative shares or shares above 100 percent. When the endpoint gap is zero, shares of the gap are undefined even if large offsetting contributions exist. Do not force a pie chart in either case.

Optional order independent allocation

After the ordered bridge is correct, consider exact Shapley allocation for at most five independently switchable rule groups. Evaluate every valid combination of baseline and target rules, at most 32 combinations, and average each rule's marginal effect across all possible orders. Cache each unique combination result.

This allocation is independent of the order chosen for presentation, but depends on the chosen groups and admissible counterfactuals. It is an attribution convention, not causal proof. Require every needed hybrid to be valid; do not omit invalid combinations and still label the result exact Shapley.

If there are more groups, use the ordered method or decline this feature. A later sampling approximation would need a sampling seed, error assessment and a clear approximate label. It is not part of the MVP.

Semantic drift

Create a new definition version whenever source text changes. Compare both raw and normalized fingerprints. A raw-only cosmetic change remains in history without a meaning-change alert. A supported change to the expression, business predicate or time field creates a definition-change event even if the observed amount does not change.

A meaning-change alert depends on the reviewed intent. If the owner intentionally changed the definition and approved its effective date, classify it as an approved change. An unapproved change against an expected baseline is suspected semantic drift. Without enough source context, use Unknown meaning impact.

Record observed_at, optional source_changed_at and optional effective_from separately. "Changed 12 days ago" is permissible only when source history supports that timestamp. Otherwise say "First observed 12 days ago." Compare old and new definitions on the same snapshot to isolate the definition effect.

For a historical replay, freeze a full extraction with an as-of cutoff and run both versions for each chosen month. Label results as restated under that cutoff. They are not the original figures published at the time unless dated historical snapshots are available. Require both definitions to be evaluable for each period; expose gaps rather than joining missing periods into a continuous line.

Separating definition and data changes

When both the definition and data change, use a two-by-two experiment if both data snapshots exist: old definition on old data, new definition on old data, old definition on new data, and new definition on new data. A selected path can separate definition and data contributions, while a Shapley-style average can share the interaction. Label the chosen method.

The MVP fixes the data and changes the definition. It must not call a difference between an old reported number and a newly calculated number a pure semantic drift impact. Changes to late-arriving records, refund cutoffs or mappings are data changes even when the SQL is identical.

5 Architecture and deployment

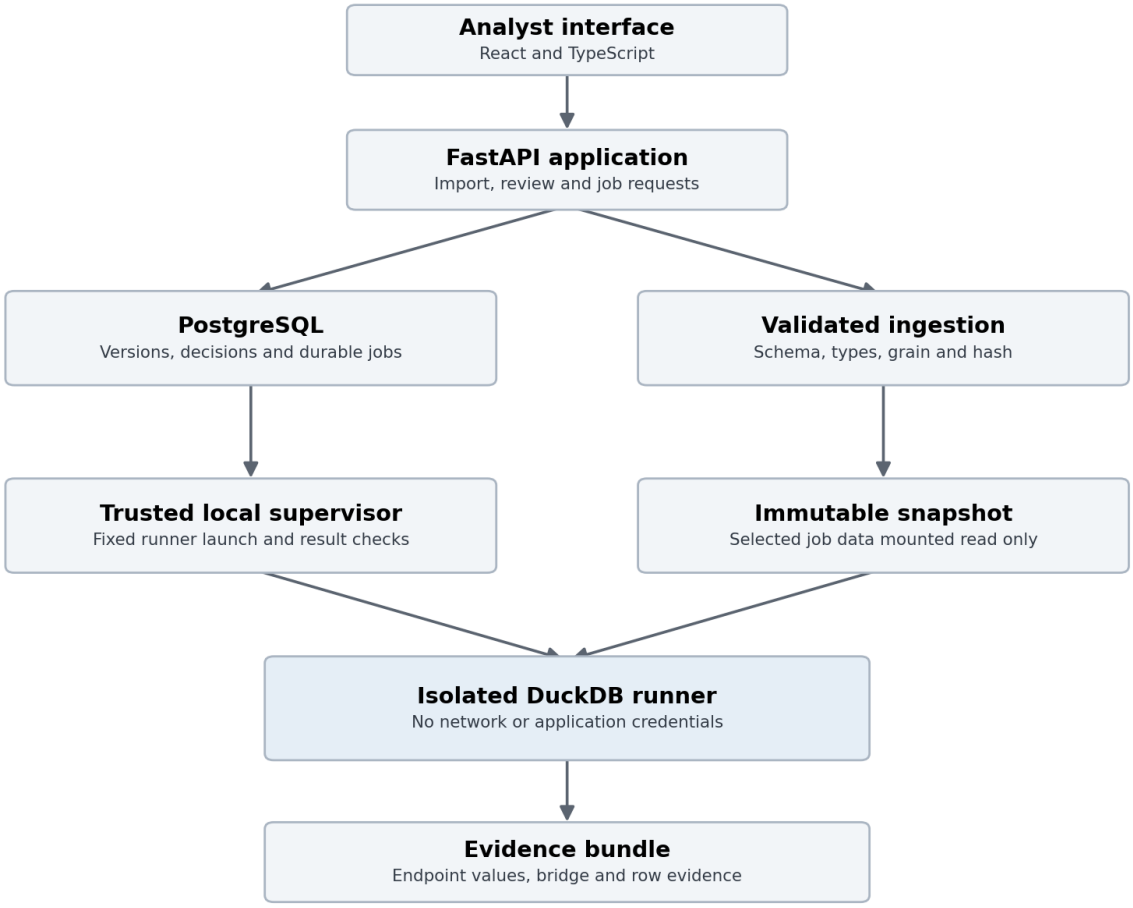
Recommended stack

Layer	Choice	Reason
Web interface	React with TypeScript and Vite	Clear typed investigation screens without unnecessary server rendering
API	Python with FastAPI and Pydantic	Validation, typed contracts and a Python analytics ecosystem
SQL interpretation	SQLGlot with a versioned restricted IR	Structural parsing and comparison with an explicit support policy
Analytical execution	DuckDB in a restricted runner	Repeatable local queries against frozen analytical snapshots
Application records	PostgreSQL	Transactions for versions, runs, claims and decisions
Data and evidence files	Local managed directories initially	Cheap reproducible storage; abstraction permits object storage later
Data preparation	DuckDB plus PyArrow where needed	Typed CSV and Parquet ingestion without maintaining two dataframe stacks
Tests	pytest, property tests and Playwright	Numeric invariants, parser behavior and one complete user journey
Packaging	Locked Python and JavaScript dependencies with containers	Repeatability across development machines
AI	No required provider in the MVP	Baseline behavior is independently testable and available offline

Pandas or Polars may be added for a specific measured need. Using both by default duplicates transformations and increases disagreement risk. PostgreSQL stores application metadata; DuckDB calculates analytical results. They have different responsibilities. Redis, Kafka, Kubernetes, a graph database and a vector database are unnecessary for the initial workload.

Select supported stable dependency versions at implementation time, verify compatibility and lock them. This plan specifies behavior rather than claiming a particular version number is the newest.

Component boundaries



MetricGuard component architecture

The browser communicates only with the API. The API handles imports, context validation, pair review and job creation. PostgreSQL stores durable job and application state. A trusted local worker supervisor claims queued jobs and launches the analytical runner. The runner receives only a validated job description and the selected snapshot, calculates evidence, and returns a bounded result bundle.

Original SQL is parsed and rejected if unsupported. Only SQL regenerated from the validated IR is executed. This reduces the exposed query language but does not replace sandboxing. DuckDB's own guidance treats untrusted SQL as executable code and recommends isolation plus resource controls [S8].

The API never receives a container engine socket. The local supervisor, started by the developer, may use container privileges to launch a fixed runner image. This privilege is confined to the trusted local orchestration process; uploaded text cannot choose images, mounts, commands or flags. In a hosted pilot, replace that mechanism with a restricted job service before allowing arbitrary remote uploads.

Execution sequence

1. API validates version IDs, context and permissions for the local project, then inserts a queued run transactionally.

2. Supervisor claims one job with a lease and an attempt number. The initial deployment permits one analytical run at a time.
3. Supervisor prepares a job-specific directory containing an immutable snapshot copy or read-only reference and a validated JSON manifest.
4. A fixed runner container starts with no network, no application credentials, no container socket and no unrelated data mounts.
5. Runner verifies hashes and policy versions, validates the IR, opens the selected prebuilt database read-only and executes the permitted endpoints and bridge.
6. Runner writes bounded JSON and evidence files to a job-specific output directory.
7. Supervisor validates output sizes, hashes, schemas and reconciliation invariants, then commits the result pointers and terminal state.
8. API serves results and paginated evidence. The browser polls status; streaming is optional later.

Ingestion is a separate operation with its own limits. It turns an accepted flat file into an immutable analytical database before untrusted metric execution begins. Do not require external file scans from the SQL runner. A snapshot is published only after ingestion validation completes.

Modules and interfaces

Module	Input	Output
Import adapter	SQL text and source metadata	Definition draft and provenance
Snapshot ingester	CSV or Parquet with declared schema	Validated immutable snapshot manifest
Parser and binder	SQL, dialect and schema	Parsed tree or structured parse error
Support checker	Parsed tree and policy version	Typed IR or unsupported reasons
Match suggester	Names, metadata and IR features	Ranked candidates with reason fields
Context validator	Definition pair and run context	Comparable status and missing-context list
Diff engine	Two IR versions	Structured rule changes
Executor	Approved IR, context and snapshot	Typed endpoint values and execution diagnostics
Reconciler	Endpoint data and valid rule switches	Row contribution evidence and ordered bridge
Drift service	Definition versions and approved scope	Version event and optional fixed-data replay
Evidence exporter	Immutable completed or partial run	Human-readable report and machine-readable bundle

Define these boundaries before creating UI pages. A single Python service package can own all modules initially; separating interfaces does not require separate microservices.

Job behavior

Lifecycle: queued, running, succeeded, partial, failed, canceled. A job stores started_at, completed_at, heartbeat_at, lease_expires_at, attempt and error_code. An expired worker lease can be reclaimed with a new attempt. Only the current lease owner can finalize that attempt. Results are written under attempt-specific temporary paths and committed atomically.

An idempotency key on run creation prevents a browser retry from creating duplicate jobs. A unique key belongs to a project and the normalized request. Reusing it with a different request returns a conflict. Cancellation sets a requested flag; the supervisor terminates the runner and records the final state. Never mark a job succeeded merely because its process exited with code zero.

A run can be partial when endpoint results exist but a bridge exceeds a limit or cannot be constructed. Endpoint execution failure makes the value comparison unavailable. Parsing and comparability errors should be resolved before queuing whenever possible.

Reproducibility and caching

The cache key includes both definition version hashes, the snapshot manifest hash, mapping version, context, IR policy version, engine version, decomposition method and configuration. A name or timestamp is insufficient. Changing any semantic input produces a new run identity.

Reproducibility means the same numeric results and evidence under fixed inputs. Timings, generated timestamps and unordered source record order need not be byte-identical; sort exported evidence by stable key and separate volatile metadata. Preserve decimal formatting and code-version references in the report.

Do not cache a failed or incomplete result as a full success. Cached results keep their original provenance. A new UI label or owner decision can reference an old run but cannot change the stored historical conclusion.

Local and hosted deployment

The recommended development environment is a laptop with approximately 16 GB RAM and Docker support, using WSL2 if building on Windows. This is a planning assumption, not a measured minimum. The web service, API and PostgreSQL can run in a local Compose setup; a documented local supervisor starts the restricted job containers. Bind development services to loopback by default.

The first shareable demonstration can be a recording or a web interface using precomputed synthetic evidence. It does not need a public SQL-execution endpoint. A private pilot with uploads requires authentication, project authorization, isolated jobs, managed secrets, retention controls and backups. Add those when moving beyond local use, without pretending the local owner labels already provide them.

Deployment is a separate decision from building the project. No deployment or external integration is required by this planning pack. A container-based backend needs an appropriate execution host; do not assume a static hosting platform can run the analytical worker.

Configuration and operation

Configuration includes database URL, artifact root, runner image digest, upload limit, query deadline, run deadline, worker concurrency, memory budget and log level. Keep secrets outside source control. Persist a redacted configuration fingerprint on each run.

Provide health checks for the API and database, a worker heartbeat, a run queue count and an artifact integrity check. Log structured error codes, durations and row counts without raw query text or row contents by default. Maintain a developer runbook for a stuck job, failed ingestion, invalid snapshot and replay failure.

6 Data model and API contracts

Relational model

Use UUID identifiers in the implementation; fixture labels in this pack are readable examples. All primary objects belong to a project even in the single-user MVP. Project scoping does not itself make the application a secure multitenant service.

Entity	Important fields	Key constraints
project	id, name, timezone, settings_version, created_at	Owns every import, run and decision
source_asset	id, project_id, kind, label, locator, content_hash, imported_at	Locator is display metadata, not an executable path
dataset_snapshot	id, project_id, manifest_hash, schema_json, key_columns, cutoff_at, artifact_ref, status	Immutable after validation; unique manifest identity
metric_family	id, project_id, name, purpose, owner_label	Family membership is reviewed, not solely inferred
metric_definition	id, project_id, family_id nullable, source_asset_id, stable_source_key	Stable implementation identity across versions
definition_version	id, definition_id, version_number, raw_sql, dialect, raw_hash, ir_json, ir_hash, support_state, observed_at	Unique definition and version number; immutable
pair_review	id, left_definition_id, right_definition_id, relation, reason, actor_label, created_at	Canonical pair ordering; decisions append or supersede
comparison_run	id, project_id, version_a, version_b, snapshot_id, context_json, method, state, idempotency_key, lease fields	Inputs immutable after queueing; unique idempotency key per project
run_result	id, run_id, endpoint_values, delta, status_fields, residual, engine_versions, evidence_ref	One accepted terminal result per attempt; decimal values stored exactly
bridge_step	id, run_result_id, position, rule_group, ir_hash, previous_value, next_value, contribution	Unique result and position; ordered contributions
baseline_approval	id, family_id, version_id, scope_json, effective_from, reason, actor_label, supersedes_id	No conflicting active approval for the same declared scope
dependency_edge	id, project_id, upstream_asset, downstream_asset, origin, observed_at, confidence_label	Known, declared or inferred origin retained
drift_event	id, definition_id, old_version, new_version, change_kind, approval_state, replay_run_id	Old and new versions fixed
audit_event	id, project_id, event_type, actor_label, target_type, target_id, before_ref, after_ref, created_at	Append-only through the application interface

Use foreign keys and transactional state transitions. Large row evidence stays in a bounded Parquet or JSON artifact, with its hash and schema referenced from the database. A result should never rely on a mutable filesystem path without an integrity record.

Versioning and retention

Importing identical source bytes creates a new import event if useful but should not create a fake semantic version change. Importing a changed definition produces a new immutable version. A changed parser or normalization policy may generate a new derived analysis; do not overwrite the old IR used by completed runs.

A baseline approval points at one immutable definition version. Editing that definition creates a new version and requires a new decision for the applicable scope. Approvals are historical records even if superseded. Reports list whether their baseline was approved at run time.

Snapshot deletion should explicitly mark dependent runs as no longer replayable while preserving their recorded results and hashes, unless the user chooses to delete those results too. Do not silently leave a reproducibility badge after source evidence is removed.

API conventions

Proposed base path is `/api/v1`. The API uses JSON except file upload and downloads. Monetary values are decimal strings. Dates are ISO 8601; timestamps include offsets. List endpoints use cursor pagination. Errors include a stable code, readable message, affected field and correlation ID. Never expose local paths, database secrets or raw stack traces in the browser.

Method and path	Purpose	Important behavior
POST /projects	Create a local project	Returns its configuration version
POST /projects/{id}/snapshots	Upload a dataset and ingestion metadata	Returns validation job; rejects size or format violations
GET /snapshots/{id}	Read schema and validation result	No accepted snapshot before validation succeeds
POST /projects/{id}/definitions	Import SQL and provenance	Returns support status and structured errors
POST /definitions/{id}/versions	Add a new immutable version	Does not update completed runs
GET /projects/{id}/definitions	List inventory and support coverage	Paginated with server-side filtering
GET /projects/{id}/candidates	List ranked potential pairs	Includes heuristic reasons and review state
POST /projects/{id}/pair-reviews	Record intended relationship	Requires rationale for intentional variant or rejection
POST /comparisons/validate	Check proposed pair and context	Does not execute SQL
POST /comparisons	Queue a validated comparison	Requires Idempotency-Key; returns 202 with run ID
GET /comparisons/{id}	Read state and results	Partial results retain limitations

Method and path	Purpose	Important behavior
GET /comparisons/{id}/evidence	Page or download evidence	Applies export limit and safe filename rules
POST /comparisons/{id}/cancel	Request cancellation	Idempotent for a terminal run
GET /definitions/{id}/history	Read versions and drift events	Distinguishes source time from observation time
POST /families/{id}/baselines	Record a scoped baseline	Expected current approval version prevents stale overwrite
POST /projects/{id}/dependencies	Register a known consumer link	Records manual origin and coverage limits
POST /comparisons/{id}/exports	Generate report and JSON bundle	Preserves incomplete and unknown result statuses

The path list is a proposed interface specification, not a generated production OpenAPI document. During implementation, generate OpenAPI from validated server models and check that the UI uses those same contracts.

Comparison request

The illustrative request in `examples/comparison_request.json` references two immutable versions, one snapshot and an August 2026 half-open interval. It names the ordered bridge method and fixes the rule order. Date parameters are bound values; no client-supplied SQL fragments are appended to queries.

Required context fields are snapshot_id, start, end_exclusive, timezone, unit, currency, grain and scope. Each definition's time field remains in that definition. The shared reporting window stays the same while attribution fields may differ.

The server checks that both versions and the snapshot belong to the same project. It rejects unknown or unsupported versions, contradictory context and incompatible scopes. A currency string on the request cannot override a snapshot's currency without an explicit supported mapping.

Result contract

The result separates relationship_status, comparability_status, definition_status, value_status and explanation_status. It includes baseline and target decimal values, signed delta, absolute gap, baseline-relative change when defined, tolerance, ordered contributions, residual and artifact references.

Provenance includes source text hashes, IR hashes, snapshot manifest hash, context fingerprint, mapping version, parser version, DuckDB version, application revision and decomposition method version. A display summary is derived from these fields. The JSON example contains readable identifiers and fixture results; actual hashes are generated from real bytes during implementation.

Error taxonomy

Code	Meaning	User action
PARSE_ERROR	SQL cannot be parsed with the declared dialect	Correct SQL or dialect
UNSUPPORTED_ CONSTRUCT	Query uses a feature outside the declared subset	Use a supported extract or defer the case
UNKNOWN_COLUMN	Definition cannot bind to the snapshot schema	Correct mapping or select another dataset
CONTEXT_REQUIRED	Period, timezone, grain or scope is missing	Supply the missing context
NOT_COMPARABLE	Known scopes, units or data are incompatible	Resolve mismatch before interpreting impact
SNAPSHOT_INVALID	Data fails declared structural rules	Inspect ingestion report
UNDEFINED_ENDPOINT	An aggregate result is NULL or otherwise undefined	Inspect empty population or missing terms
EXECUTION_LIMIT	Time, memory, disk or row output limit reached	Narrow the case or revise configured limits
RECONCILIATION_ MISMATCH	An invariant failed	Inspect evidence; report cannot claim full explanation
STALE_REVIEW	Baseline changed since the review was opened	Reload and review the current state

Use HTTP 400 for malformed requests, 413 for size violations, 422 for typed semantic validation failures, 404 for unavailable objects and 409 for idempotency or version conflicts. A queued job can later fail with a structured domain error without turning its status endpoint into an HTTP server failure.

7 Security and operational requirements

Trust boundaries

The MVP is a local single-user application for imported extracts and synthetic demonstrations. Uploaded files, SQL, descriptions, filenames and any future model output are untrusted inputs. The API, parser, supervisor and runner image are trusted application components. The runner receives no production credentials and has no network access.

Do not rely on a SELECT-only string check. A SELECT can call functions that read files or perform expensive work. Enforce a typed allowlist across the whole parsed structure, registered table references and approved expressions, then regenerate executable SQL from the IR. Bind period values separately and quote validated identifiers through the SQL library.

DuckDB settings can reduce capabilities but are not a complete sandbox [S8]. Apply external-access restrictions, disable extension installation and autoloading, lock configuration after setup, and open the prepared database read-only. Also restrict the process at the container and host level. Do not claim read-only access alone prevents file reads or resource exhaustion.

Initial limits

These are proposed conservative defaults to validate during the engineering spike. They are not measured scale claims.

Resource	Proposed initial limit	Failure behavior
Uploaded analytical file	25 MiB	Reject before ingestion; explain the configured limit
Imported definitions per project	100	Ask user to remove or split the import
SQL definition length	50000 characters	Reject before parse
Snapshot rows	1000000	Reject or request a smaller extract; no silent sampling
Analytical concurrency	One job	Queue additional jobs
Per-query wall time	15 seconds	Terminate execution and mark partial or failed
Total run wall time	60 seconds	Stop remaining work and preserve honest status
Runner memory	512 MiB DuckDB target within a 1 GiB container limit	Terminate on hard limit; avoid claiming engine limit covers all memory
Output rows displayed	100 per page	Cursor pagination
Evidence export size	100000 rows or 25 MiB	Offer aggregate evidence and mark full-row export limited
Rule groups in ordered bridge	Five	Decline more complex attribution for MVP

The normal performance target of 30 seconds applies to the standard 100000-row fixture under the named development hardware. The larger accepted maximum is a hard admission limit, not a promise of the same latency. Record query count and total runtime so expensive decompositions remain visible.

File and runner controls

Use generated filenames and a managed root, not user-supplied paths. Check MIME and extension against actual parser behavior. Reject archive uploads initially to avoid path traversal and decompression complexity. CSV and Parquet readers need type, size and structural validation. Malformed files should be isolated during ingestion just like queries during execution.

Run the fixed analytical image as a non-root user with a read-only root filesystem, dropped capabilities, a bounded temporary directory, process and CPU limits, and no network. Mount only that job's snapshot read-only and a dedicated output directory with a quota or equivalent host-enforced bound. The supervisor must validate result size and schema before reading it into application memory.

The local supervisor can use container launch privileges but has no remotely exposed arbitrary-command interface. Its launch template, image digest and mount roots are application-controlled. If this isolation cannot be reliably provided on the development machine, use the

precomputed demo mode until the runner is fixed; do not quietly run uploaded SQL in the API process.

Data minimization and exports

Use synthetic order and customer IDs for the portfolio. Real names, emails and payment details are unnecessary for reconciliation. Row evidence defaults to IDs, membership flags and numerical contributions. An explicit user choice is required before adding other columns to an evidence export.

Do not send source records or SQL literals to an AI provider in the MVP. If an optional metadata matcher is later enabled, document what fields leave the machine and provide an offline alternative. Descriptions and comments remain data even when they contain instructions addressed to a model.

Escape HTML in displayed source text and reports. Make spreadsheet-compatible CSV exports safe from formula injection by treating untrusted text beginning with formula characters as text. Keep the exact machine-readable evidence separately so display escaping does not change underlying values.

Authentication and review

The local version uses a self-declared owner label and loopback-only access. It should say Recorded locally instead of Authenticated approval. Before a network-accessible private pilot, add identity, project-level authorization on every object, protection against cross-project references, secure sessions and download authorization.

Reader, analyst and owner roles are a later deployment concern. Readers can view approved evidence, analysts can import and investigate, owners can record scoped baselines. Do not invent an enterprise compliance badge or claim an audit log is tamper-proof; the local database administrator can modify it.

Retention and recovery

Default local retention can preserve snapshots and evidence until the project owner deletes them. Show disk usage and allow deleting a project or snapshot with clear dependent-run consequences. A private pilot should define a stated retention period with its users rather than inherit unlimited retention accidentally.

Back up metadata and artifact files together with a manifest of hashes. Test one restore before using real pilot data. A database-only backup cannot reproduce runs whose snapshots were lost. Clean abandoned temporary run directories after a defined interval while retaining accepted result artifacts.

Operational runbook

For a stuck job, inspect the worker heartbeat and lease, terminate the expired runner if it still exists, and requeue only through an explicit new attempt. For a corrupted snapshot, block new runs and ask for reimport; preserve existing reports with an integrity warning. For a parser upgrade, rerun the regression corpus before changing the policy version used for new analyses.

Monitor run states, failure codes, parse coverage, context-blocked cases, queue wait, execution time, output size and reconciliation failures. Avoid raw-data logging. A successful API health

check says nothing about whether the analytical runner can complete a job, so include a bounded synthetic health fixture in operational checks when needed.

8 Demo data and evaluation plan

Dataset design

Use an invented online retailer and a documented order-level dataset. The eight-row golden fixture below establishes the worked example by hand. A separate seeded generator can later create 100000 orders over twelve months for performance and history demonstrations. Synthetic scale data must not replace edge cases or independent expected results.

The golden fixture is stored as `examples/orders_golden.csv`. All amounts are INR. order_amount excludes tax and shipping. refund_amount is the total refund attached to that order as of the declared extraction cutoff, rather than a refund-event ledger. Dates are business dates in Asia/Kolkata. The August window is 1 August inclusive through 1 September exclusive. The extraction cutoff is 2 September 2026.

ID	Order date	Payment date	Status	Payment	Amount INR	Refund INR
O01	2026-08-05	2026-08-06	Completed	Paid	60000000	2400000
O02	2026-08-10	2026-08-11	Completed	Paid	40000000	1920000
O03	2026-08-15	2026-08-16	Completed	Paid	2320000	0
O04	2026-08-20	NULL	Completed	Unpaid	420000	0
O05	2026-08-31	2026-09-01	Completed	Paid	1260000	0
O06	2026-08-17	NULL	Pending	Unpaid	700000	0
O07	2026-08-14	NULL	Cancelled	Unpaid	500000	0
O08	2026-08-14	NULL	NULL	Unpaid	200000	0

The unusually large individual orders keep the dataset small while matching the original crore figures. They are not intended as a realistic retail transaction distribution. The larger synthetic dataset should use a plausible documented distribution and report its own independently calculated totals.

Three explicit definitions

Dashboard A uses `SUM(order_amount)`, `status = 'Completed'`, and the August period on order_date. Dashboard B uses `SUM(order_amount)`, `status <> 'Cancelled'`, and that same order-date period. Finance uses `SUM(order_amount - COALESCE(refund_amount, 0))`, `payment_status = 'Paid'`, and the August period on payment_date. These queries are included in the examples folder with explicit date literals.

Finance's COALESCE is an explicit policy choice that treats a NULL refund as zero. The golden rows do not contain a NULL refund, so a separate test is needed to expose the effect. The NULL status in O08 passes neither equality nor inequality predicates because SQL uses unknown rather than true for those comparisons.

Expected results

Assertion	Expected result
Dashboard A total	INR 104000000.00
Dashboard B total	INR 104700000.00
Finance total	INR 98000000.00
Finance minus A	INR -6000000.00
Finance versus A relative change	Approximately -5.769230769 percent
B minus A	INR 700000.00
Refund bridge contribution	INR -4320000.00
Eligibility bridge contribution	INR -420000.00
Time bridge contribution	INR -1260000.00
Full bridge residual	INR 0.00

The four nonzero order contributions for Finance minus A are O01 at -2400000, O02 at -1920000, O04 at -420000 and O05 at -1260000. O03 contributes zero difference. O06 through O08 are excluded from both A and Finance. Dashboard B's additional amount comes from O06 alone.

The bridge deliberately changes eligibility before time attribution. If time is changed first, the unpaid O04 with a NULL payment date leaves during that step; some of the attribution moves from eligibility to time. Endpoint totals do not change. Include this as a demo of why an explanation method must be named.

Schema dictionary

Field	Type	Meaning and validation
order_id	String	Unique non-NULL row key
customer_id	String	Synthetic entity ID; distinct-count fixtures may contain repeated customers
order_date	Date	Business order date; non-NULL in the golden fixture
payment_date	Nullable date	Business payment date; NULL means not recorded
status	Nullable string	Completed, Pending, Cancelled or explicitly unknown
payment_status	String	Paid or Unpaid in the golden fixture
order_amount	Decimal 18 2	Single-currency amount excluding tax and shipping
refund_amount	Nullable decimal 18 2	Order-level accumulated refund as of cutoff
currency	String	INR only for the golden snapshot

The production-like test set should add duplicates, missing IDs, invalid dates and currency mismatches as separate invalid snapshots. Do not mix those into the validated golden fixture and silently clean them.

Test matrix

ID	Scenario	Required result
T01	Golden A versus Finance	Correct endpoints, four nonzero rows and complete ordered bridge
T02	A versus B with pending order	Predicate difference and INR 700000 extra
T03	Different predicates with no pending rows	Values equal, definition still different
T04	Whitespace and output alias change	Cosmetic version event without meaning alert
T05	NULL status with inequality	Row excluded according to SQL semantics
T06	NULL refund with and without COALESCE	Different term behavior retained; no implicit cleanup
T07	No matching rows	SUM is undefined, COUNT is zero; no forced money gap
T08	Zero baseline and nonzero target	Absolute gap shown; relative percentage undefined
T09	Opposing rule effects	Signed contributions shown without forcing 0 to 100 shares
T10	Refund and date interaction on one row	Bridge changes by order; endpoint and row sums remain stable
T11	Duplicate order_id	Row evidence blocked and invalid snapshot reported
T12	Mixed currencies	Impact blocked until a supported mapping exists
T13	Incompatible period or timezone	Needs context or not comparable
T14	Join, CTE, window or division expression	Unsupported status rather than partial silent parsing
T15	File scan or second SQL statement	Rejected before execution
T16	Oversized file or resource exhaustion	Bounded failure and honest job state
T17	Worker dies mid-run	Lease recovery creates a new attempt without two accepted results
T18	Repeated request with same idempotency key	One run returned; changed payload receives conflict
T19	Same inputs and versions repeated	Numeric results and stable evidence reproduce
T20	Snapshot or engine version changes	New run identity; no stale cache reuse
T21	Changed definition with unchanged data	Fixed-data drift effect correctly isolated
T22	Same definition with changed data	Data change not labeled pure semantic drift

ID	Scenario	Required result
T23	Duplicate customer across orders	COUNT DISTINCT uses entity membership, not sum of order counts
T24	Missing snapshot after deletion	Historical result visible, replay unavailable
T25	Partial attribution or invalid hybrid	Endpoint remains visible; explanation cannot be complete
T26	SQL or description contains HTML or instructions	Escaped text; no model or runner instruction following

Independent oracles and evaluation

Verify the golden totals using a separate decimal calculation and explicit membership lists, not by calling the same parser and reconciler under test. For generated datasets, independently calculate simple expected aggregates and keep fixed seeds. Use property tests for sign reversal of endpoint deltas, identity comparisons, row partition coverage and bridge telescoping.

Do not require each ordered rule contribution to reverse exactly when the direction changes; interactions and the declared sequence can change attribution. The endpoint gap must reverse. Identity comparison must produce a zero gap for defined endpoints and preserve undefined endpoint semantics for empty SUM cases.

Create a matching corpus with at least 100 labeled pairs covering expected agreement, intentional variants, unrelated names and unknown cases. Split it by business concept or template family so near-duplicate definitions do not leak between tuning and evaluation. Report precision, recall, confusion cases and the abstention rate; a high precision score with almost no suggestions can be unhelpful.

Five minute demonstration

1. Show the three revenue totals and ask which number should be used for which purpose.
2. Open the definition matrix and show status, refund and date differences.
3. Run A against Finance on the frozen August snapshot and show the INR 60 lakh gap.
4. Open the bridge and click through to the four contributing orders.
5. Compare A with B to show that not Cancelled includes a pending order.
6. Show a different-definition but equal-value case to demonstrate that equality is not proof.
7. Import a changed date rule, replay it on the same snapshot, and record a scoped baseline decision.
8. Export evidence with the limitations and reproducibility identifiers intact.

The strongest portfolio proof is this reproducible investigation plus tests and a clear explanation of limits. A screenshot of a dashboard alone does not demonstrate the reconciliation engine.

9 Backlog and delivery roadmap

Effort and sequence

The proposed core build requires approximately 150 to 205 focused hours for one developer who can work with Python, SQL and a basic React application. At 15 to 20 hours per week, that is roughly 8 to 14 weeks before contingency. Add 20 to 30 percent for learning unfamiliar tools and resolving integration issues. These are planning estimates, not delivery promises.

Milestone	Scope	Estimated hours	Exit evidence
M0 Feasibility	Golden fixture, grammar decisions, isolated execution spike	12 to 16	Independent expected totals and supported-subset decision
M1 Core comparison	IR, parser support checker, context and endpoints	28 to 36	Golden and unsupported fixtures pass
M2 Explanation and history	Row evidence, ordered bridge and version diff	30 to 40	Reconciliation invariants and drift replay pass
M3 Application layer	Metadata model, jobs, API and artifact persistence	24 to 32	Interrupted job recovery and reproducible saved results
M4 Analyst interface	Imports, inventory, comparison, history and export	32 to 44	Complete user journey without direct database intervention
M5 Evaluation and presentation	Regression corpus, performance checks, pilot, docs and recording	24 to 37	Evidence pack, limitations and measured benchmark

Build one vertical investigation before implementing every inventory or settings screen. If the project is too large, retain SUM comparisons, row evidence, the ordered bridge and definition history; defer automated suggestions, COUNT DISTINCT and baseline workflow polish. Do not cut the honesty of unsupported and partial states.

Build backlog

ID	Priority	Work item	Depends on	Done when
MG01	Must	Freeze data dictionary and golden fixture	None	Totals verified independently
MG02	Must	Define typed IR and support policy	MG01	Every supported syntax maps unambiguously
MG03	Must	Implement parser and schema binding	MG02	Good and bad definitions have stable outcomes
MG04	Must	Validate and freeze imported snapshots	MG01	Content hash, schema and key validation recorded
MG05	Must	Build restricted runner	MG02 MG04	Untrusted scans and resource escapes are blocked in tests

ID	Priority	Work item	Depends on	Done when
MG06	Must	Validate comparison context	MG03 MG04	Missing or incompatible context blocks impact
MG07	Must	Compute endpoint values	MG05 MG06	Golden scalar values match the oracle
MG08	Must	Produce additive row evidence	MG07	Row differences sum to endpoint gap
MG09	Must	Build ordered rule bridge	MG03 MG07	Every intermediate stored and residual checked
MG10	Must	Classify version differences	MG03	Cosmetic, structural and unknown changes separated
MG11	Must	Store immutable versions and runs	MG02	No edit alters previous evidence
MG12	Must	Add durable jobs and recovery	MG05 MG11	Duplicate requests and dead workers handled
MG13	Must	Expose typed API contracts	MG06 MG11 MG12	UI contract agrees with server validation
MG14	Must	Build comparison screen	MG08 MG09 MG13	User can inspect exact gap and evidence
MG15	Must	Build import and inventory screens	MG03 MG04 MG13	Support and coverage are explicit
MG16	Must	Build history and fixed-data replay	MG10 MG11 MG13	Definition changes replay on fixed data
MG17	Should	Add deterministic candidate ranking	MG03 MG11	Evaluated suggestions with manual review
MG18	Should	Add scoped baseline decisions	MG11 MG13	Approval points to immutable version and scope
MG19	Should	Register report dependencies	MG11 MG13	Known consumer list shows evidence and coverage
MG20	Must	Export evidence and provenance	MG08 MG09 MG11	Report and JSON retain all limits
MG21	Must	Complete regression and end-to-end tests	Core milestones	Defined release gates pass
MG22	Must	Prepare demo and documentation	MG14 MG16 MG20 MG21	Another person reproduces the golden investigation
MG23	Later	dbt manifest metadata adapter	Stable core	Versioned artifacts import without executing macros
MG24	Later	Power BI TMDL metadata adapter	Stable core	Import capability clearly separated from execution
MG25	Later	Exact bounded Shapley allocation	MG09	All combinations valid and results tested

ID	Priority	Work item	Depends on	Done when
MG26	Later	Optional metadata embeddings	MG17	Improves held-out evaluation over baseline

"Must" describes the full proposed MVP. The reduced first demonstration may omit Should items and COUNT DISTINCT support while explicitly reporting its smaller capability set. Later items must not be quietly bundled into a first coding prompt.

First engineering spike

Before building application screens, prove four things. First, the parser can reliably distinguish the three permitted rule groups and reject unsupported clauses. Second, generated execution queries preserve the tested NULL and decimal behavior. Third, the chosen local environment can start and terminate an isolated runner with no network. Fourth, row contributions and the bridge agree with the independent golden calculation.

Timebox the spike to M0. If arbitrary imported SQL cannot be decomposed reliably, reduce the grammar and explain its support boundary. Do not compensate with an LLM that guesses the missing logic. If isolation is not ready, keep the user interface in precomputed demonstration mode until it is.

Definition of done

Each feature has documented inputs, failure states and a tested acceptance condition. Numeric features include independent expected results or invariants. A feature involving a new query construct includes a negative case and a counterexample for meaning-preserving transformations. UI work includes accessible errors and retains context when a run finishes.

The MVP is done when a fresh local setup reproduces the golden demo, unsupported queries cannot masquerade as successful comparisons, complete bridges reconcile, version history is immutable, jobs survive interruption, and a reviewer can trace every displayed amount to evidence. Do not make broad enterprise scale, security or savings claims from that milestone.

Release checklist

1. Lock runtime and package versions and record the revision used for results.
2. Run the focused regression corpus and the end-to-end golden investigation.
3. Verify snapshot, query and context hashes appear in exports.
4. Test cancellation, timeout, duplicate request and worker-restart behavior.
5. Check that unsupported, undefined and partial results remain explicit everywhere.
6. Inspect the six screens using the smallest supported layout and keyboard navigation.
7. Publish the data dictionary, support matrix, benchmark method and known limits.
8. Record the demo using synthetic data and state which integrations are actually implemented.

Budget assumptions

The local deterministic MVP requires no paid LLM API or managed warehouse. It uses the developer's machine, local storage and an open-source software stack; existing internet,

hardware and electricity costs still apply. Check the current license of each dependency before distributing the project.

Hosted cost depends on active runner minutes, stored snapshot bytes, database service choice, backups and outbound downloads. Model it from measured workload instead of choosing an arbitrary monthly price. If an AI matcher is later added, estimate cost from imported metadata count and model token usage, with a hard spending limit and caching.

10 Decisions risks and coding handoff

Architecture decision record

ID	Decision	Reason	Revisit when
ADR01	SQL first with a declared grammar	Makes evidence and limits testable	Core reconciliation passes its evaluation
ADR02	One common immutable snapshot	Separates logic impact from data changes	A validated cross-source mapping case is available
ADR03	DuckDB for analytics and PostgreSQL for records	Separates calculation from application transactions	Measured operational overhead outweighs the benefit
ADR04	Typed IR regenerated into SQL	Bounded execution and structured differences	A supported construct cannot be expressed faithfully
ADR05	Ordered bridge before Shapley	Small, inspectable and reproducible	Interactions make order sensitivity a practical user problem
ADR06	Human review of metric relationships	Similar names can describe different valid concepts	Strong evidence supports a narrowly bounded automation
ADR07	Deterministic explanation templates	Prevents fabricated causal and numeric statements	A constrained summarizer adds measured usefulness
ADR08	One worker and PostgreSQL job records	Adequate for portfolio workload	Queue measurements justify more concurrency
ADR09	Local deployment first	Keeps data access and operational scope manageable	Real pilot users require remote access
ADR10	No automatic source correction	Owners must decide intended business meaning	A separately approved reviewed-change workflow exists

Risk register

Risk	Why it matters	Mitigation or decision
Scope grows to every BI language	The core engine never reaches a credible result	Publish support levels and phase adapter work
Valid variants become false alerts	Users lose confidence quickly	Review intent before calling a definition a conflict
Hidden report filters are unavailable	Same formula can produce different visible values	Mark context incomplete; require native BI context later
Query transformation changes meaning	Explanation can be numerically neat but wrong	Typed subset, independent fixtures and no best-effort execution
Interacting rules double count impact	Percentages can be misleading	Mutually exclusive row partitions and a named ordered bridge
Mutable or incomplete snapshots	Replays cannot isolate definition effects	Freeze data, record cutoff and validate scope
Untrusted SQL reaches the host	File access or resource exhaustion	Restricted IR plus isolated runner and hard limits
A polished demo hides unsupported cases	Portfolio claims become hard to defend	Demonstrate an unsupported and an equal-value case
Market gap is overstated	Product differentiation lacks evidence	Interview users and describe tested workflow, not uniqueness
Schedule depends on unfamiliar React or infrastructure	A solo build can expand substantially	Budget learning contingency and prioritize the numeric core

Defaults selected for this plan

No user clarification is required to start the bounded prototype. The default domain is online retail, the first language is DuckDB SQL, the first currency is INR, the demonstration timezone is Asia/Kolkata, and the first deployment is local. The main calculation is additive sales reconciliation, followed by order and customer counts. Source data is synthetic.

These are reversible product choices. Revisit them if an actual user's sanitized example demands a different domain or source. Preserve the core goal of evidence-backed reconciliation rather than adding fashionable tools to the stack.

Questions to resolve through evidence

Ask potential users whether their disputed KPI should agree across reports or serves intentionally different purposes. Determine whether they can obtain source definitions and a comparable extract, whether hidden filters are common, and whether they primarily need a one-off reconciliation or ongoing monitoring. Ask what evidence they would need before changing a metric definition.

During the spike, resolve the acceptable SQL grammar, runner isolation on the target laptop, precision and empty-result handling, and the measurable latency of the standard fixture. Before

a real pilot, resolve authentication, access boundaries, retention, data handling and the person responsible for approving a baseline.

Repository plan

Use a single repository containing `apps/web` for the frontend, `services/api` for the HTTP interface, `services/supervisor` for trusted job orchestration, `packages/metricguard_core` for parsing and reconciliation, `schemas` for contracts, `tests` for golden and negative fixtures, `demo` for synthetic data, `docs` for this plan, and `infra` for local packaging.

Keep the analytical core importable independently of FastAPI and React. The first tests should exercise it without a browser or application database. This is a directory plan, not a request to create the implementation during the planning stage.

Initial coding brief

When implementation begins, use the following brief as the first bounded task. The instructions describe intended future work, not actions already performed.

> Build MetricGuard milestone M0 and the smallest part of M1 needed to validate it. Read the product scope, SQL grammar, reconciliation method and test fixture in this pack. Implement only the declared SQL subset for one frozen order dataset. Preserve original SQL, validate the full syntax tree, produce a typed IR, and calculate the three golden endpoint totals using a restricted runner. Independently verify expected values and add negative tests for unsupported syntax, NULL status, empty SUM and file access. Document the exact supported constructs and any blockers. Do not add Power BI, Excel, a warehouse connector, an LLM, automatic source edits or a full dashboard in this milestone. Finish with a reproducible golden calculation, a tested support matrix and the updated decision log.

The next task should implement row evidence and the ordered bridge only after those gates pass. Then add durable records and a minimal comparison screen. This order lets the application grow around a verified result.

Portfolio evidence and accurate claims

After the implementation is complete and verified, a defensible project description could say: "Built a SQL KPI reconciliation tool that compares versioned definitions on frozen datasets, explains additive differences with row evidence, and detects changes in metric logic." Add only the integrations and measured results actually achieved.

Use the synthetic demo as a reproducibility claim, not an assertion of real cost savings. Do not put customer counts, accuracy percentages, enterprise adoption or reduced investigation time on the resume until the corresponding evidence exists. Before implementation, describe MetricGuard as a proposed project.

The project complements an analytical portfolio by demonstrating that you can question the meaning of a KPI, design a fair comparison, calculate its impact and communicate limitations. Its distinct value should come from that work rather than from calling every component an AI agent.

11 Sources and glossary

Research notes

Sources below were reviewed on 22 September 2026. They establish the capabilities and limitations used in the design. They do not validate MetricGuard's commercial demand, uniqueness, accuracy, performance or delivery estimates. All product requirements and engineering defaults in this pack are proposed decisions.

S1 dbt Semantic Layer FAQs. The relevant documented behavior is centralized metric definition and querying with MetricFlow. This supports treating a semantic layer as an adjacent solution rather than claiming the metric-governance problem is untouched.

https://docs.getdbt.com/docs/use-dbt-semantic-layer/sl-faqs

S2 Soda documentation. Observability monitors data and metric behavior, while contracts capture declared expectations. These are relevant adjacent capabilities; this review does not establish the absence of a particular reconciliation feature.

https://docs.soda.io/data-observability

https://soda.io/product/data-contracts

S3 Datafold Monitors as Code. Data diff monitors compare datasets, and metric monitors observe standard or custom metrics. Dataset diffing therefore belongs in the competitive context.

https://docs.datafold.com/data-monitoring/monitors-as-code

S4 SQLGlot documentation and semantic diff. SQLGlot exposes parsing, expression inspection and AST differences. Its documentation describes best-effort transpilation and a strict unsupported mode. MetricGuard's business-level comparability and validated IR are additional application logic.

https://sqlglot.com/sqlglot.html

https://sqlglot.com/sqlglot/diff.html

S5 Microsoft TMDL documentation. TMDL and PBIP can expose semantic model metadata. This supports a future metadata importer and does not establish that a measure can be evaluated from its text alone.

https://learn.microsoft.com/en-us/power-bi/transform-model/desktop-tmdl-view

S6 Microsoft DAX glossary. Filter context includes filters directly applied to model columns and filters propagated through relationships. This is why the first release does not promise generic DAX-to-SQL equivalence.

https://learn.microsoft.com/en-us/dax/dax-glossary

S7 dbt manifest reference. The manifest describes project resources and dependency maps. Compiled query content is conditional rather than guaranteed for all nodes; artifact schema handling must be version-aware.

https://docs.getdbt.com/reference/artifacts/manifest-json

S8 DuckDB security guidance. Untrusted SQL requires proper isolation. Configuration limits provide additional restrictions but do not replace a sandbox.

https://duckdb.org/docs/current/operations_manual/securing_duckdb/overview

The Dview article cited in the starting concept was also reviewed as a discussion of the problem. It is not used as proof of a unique market gap or as a technical execution specification.

https://dview.io/blogs/semantic-drift-why-your-metrics-change-without-warning

Glossary

Term	Meaning in this project
KPI	A business measure used to assess performance
Reconciliation	Explaining how two comparable calculations produce different results
Metric family	A reviewed grouping of related business measures
Definition version	An immutable snapshot of one metric implementation
Semantic drift	A change in intended or implemented metric meaning over time
IR	Typed intermediate representation of the supported calculation rules
AST	The syntax tree produced by a parser from SQL text
Grain	What one row represents, such as one order
Snapshot	Frozen analytical data with recorded identity and context
Comparability	Whether the data and context support a fair comparison
Ordered bridge	A sequence of rule changes whose signed effects connect the endpoints
Residual	The part of the observed gap not accounted for by the reported bridge
Shapley allocation	An attribution convention that averages marginal effects across rule orders
Lineage	Recorded dependencies between data, definitions and reports
Canonical baseline	An approved definition for a declared purpose and scope
Idempotency	Repeating an equivalent request does not create duplicate work
Golden fixture	A small fixed example with independently established expected results
Abstention	An explicit statement that the system cannot support a conclusion

Preimplementation completion criteria

This pack covers the problem, target users, competitive context, scope, acceptance criteria, journeys, screens, SQL policy, metric contracts, calculation method, architecture, storage, APIs, security, demo data, testing, delivery sequence, risks and coding handoff. Implementation can begin with the bounded feasibility milestone. Real customer validation and measured engineering performance remain open work, not prerequisites that require inventing answers.

Appendix Explicit SQL examples

These specification fixtures use the synthetic orders table and an explicit August 2026 reporting period. They are not a general-purpose execution interface.

Dashboard A

```
SELECT SUM(order_amount) AS revenue
FROM orders
WHERE status = 'Completed'
  AND order_date >= DATE '2026-08-01'
  AND order_date < DATE '2026-09-01';
```

Dashboard B

```
SELECT SUM(order_amount) AS revenue
FROM orders
WHERE status <> 'Cancelled'
  AND order_date >= DATE '2026-08-01'
  AND order_date < DATE '2026-09-01';
```

Illustrative finance definition

```
SELECT SUM(order_amount - COALESCE(refund_amount, 0)) AS revenue
FROM orders
WHERE payment_status = 'Paid'
  AND payment_date >= DATE '2026-08-01'
  AND payment_date < DATE '2026-09-01';
```